

DSA Cheat Sheet (2026 Spring)

Updated 2026-06-02 09:30 GMT+8

Compiled by 徐前 物理学院 (2026 Spring)

说明:

- 面向机考，模板尽量精炼。
- 机考不考：KMP / Manacher / BIT / 线段树 / SMT（斯坦纳树）/ AVL / KD Tree / 倍增法。

0. Python基础

0.1 Input & Output

题型：多组数据/大输入

经典题目：OJ通用

```
import sys
data = sys.stdin.read().split()
it = iter(data)
t = int(next(it))
for _ in range(t):
    n = int(next(it))
    arr = [int(next(it)) for _ in range(n)]
```



0.2 基本数据结构

题型：队列/堆/字典

经典题目：BFS / Dijkstra

```
from collections import deque, defaultdict
import heapq

# 双端队列
dq = deque([1])
x = dq.popleft()

heap = []
heapify(heap) # transforms list into a heap, in-place, in linear time
heapq.heappush(heap, 3)
mn = heapq.heappop(heap)

item = heappushpop(heap, item)
# pushes a new item and then returns the smallest item; the heap size is unchanged
item = heapreplace(heap, item)
# pops and returns smallest item, and adds new item; the heap size is unchanged

cnt = defaultdict(int)
cnt["a"] += 1
```

0.3 二分查找

- 要求操作对象是有序序列（此处记作 lst）

```
import bisect
bisect.bisect_left(lst, x)
# 使用bisect_left查找插入点，若x∈lst，返回最左侧x的索引；否则返回最左侧的使x若插入后能位于其左侧的元素的当前索引 [1,
```

```
n]
bisect.bisect_right(lst,x)
# 使用bisect_right查找插入点, 若x∈lst, 返回最右侧x的索引; 否则返回最右侧的使x若插入后能位于其右侧的元素的当前索引 [0,
n-1]
bisect.insort(lst,x)
# 使用insort插入元素, 返回插入后的lst
```

0.4 二进制操作

基本操作: `bit_count` , `bit_length`

判断一个数 i 是否是 2 的幂, 可以使用经典的位运算技巧: `i & (i - 1) == 0`

一些有关位运算的内置函数

`int(s, 2)` 将01字符串转为对应的十进制数

`f'{n:0xxb}'` 规定二进制数的位数为xx, 不足的用0补全

```
def sortByBits(arr: List[int]) -> List[int]:
    arr.sort(key = lambda x: (x.bit_count(), x)) # 统计二进制表示中 '1' 的个数
    return arr

def countMonobit(n: int) -> int: ## find '1' '11' '111' in [1, n]
    count = 0
    for i in range(n+1):
        if (1 << i.bit_length()) - 1 == i:
            count += 1
    return count
```

0.5 OOP 速记

- 类是 ADT 的实现载体, 常见魔术方法: `__init__` / `__str__` / `__add__` 。

```
class Vec:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __str__(self):
        return f"({self.x},{self.y})"

    def __add__(self, other):
        return Vec(self.x + other.x, self.y + other.y)
```

1. 复杂度与查找

1.1 排序

快速排序

```
def quick_sort(arr, l, r):
    '''对[l, r]闭区间排序'''
    # base case
    if l == r:
        return
    # 第一步: 分成子问题
    i, j = l - 1, r + 1 # 双指针
    pivot = arr[l + r >> 1]
    while i < j:
```

```

    i += 1
    while arr[i] < pivot:
        i += 1
    j -= 1
    while arr[j] > pivot:
        j -= 1
    if i < j:
        arr[i], arr[j] = arr[j], arr[i]
# 第二步：递归处理子问题
quick_sort(arr, l, j)
quick_sort(arr, j + 1, r)
# 第三步：子问题合并（快排这一步不需要操作，但归并排序的核心在这一步骤）

```

归并排序

```

def merge_sort(a):
    if len(a) <= 1:
        return a
    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    return merge(left, right)

def merge(l, r):
    res = []
    i = j = 0
    while i < len(l) and j < len(r):
        if l[i] <= r[j]:
            res.append(l[i]); i += 1
        else:
            res.append(r[j]); j += 1
    res.extend(l[i:])
    res.extend(r[j:])
    return res

```

1.2 二分答案

题型：答案二分（单调 check）

经典题目：河中跳房子

```

# 河中跳房子 找可能的最大值
l = 1
r = L + 1
while r - l > 1: # 候选区间 [l, r - 1]
    mid = (l + r) // 2
    if stone(mid) <= m : l = mid
    else: r = mid
print(l)

```

2. 线性结构

2.1 栈 Stack

(1) 括号匹配

经典题目：03704 括号匹配问题

```
def match_brackets(s):
    st = []
    pair = {')': '(', ']': '[', '}': '{'}
    for ch in s:
        if ch in '([{':
            st.append(ch)
        elif ch in pair:
            if not st or st[-1] != pair[ch]:
                return False
            st.pop()
    return not st
```

(2) 调度场算法

要点：

- 数字直接输出。
- 运算符按优先级弹栈；遇（ 入栈，遇 ）弹到（。
- 需要前缀：反转输入 + 调度场 + 反转输出。

```
def infix_to_postfix(expr):
    pre = {'+': 1, '-': 1, '*': 2, '/': 2}
    out = []
    st = []
    for tok in expr:
        if tok.isdigit():
            out.append(tok)
        elif tok == '(':
            st.append(tok)
        elif tok == ')':
            while st and st[-1] != '(':
                out.append(st.pop())
            st.pop()
        else:
            while st and st[-1] != '(' and pre[st[-1]] >= pre[tok]:
                out.append(st.pop())
            st.append(tok)
    while st:
        out.append(st.pop())
    return ''.join(out)
```

(3) 含括号表达式求值

- 求出括号内的值后，将其压入栈

20140：今日化学论文

把连续的x个字符串s记为 [xs]，输入由小写英文字母、数字和[]组成的字符串，输出原始的字符串。

样例：输入 [2b[3a]c]，输出 baaacbaaac

```
s = input()
stack = []
for i in range(len(s)):
    stack.append(s[i])
    if stack[-1] == '[':
        stack.pop()
        helpstack = [] # 利用辅助栈求括号内的原始字符串，记得每次用前要清空
        while stack[-1] != '[':
```

```

        helpstack.append(stack.pop())
    stack.pop()
    numstr = ''
    while helpstack[-1] in '0123456789':
        numstr += str(helpstack.pop())
    helpstack = helpstack*int(numstr)
    while helpstack != []:
        stack.append(helpstack.pop())
    print(''.join(stack))

```

2.2 单调栈

题型：右侧第一个更大元素

经典题目：接雨水、护林员盖房子（最大矩形面积）

```

def next_greater_index(a):
    st = []
    ans = [0] * len(a)
    for i, x in enumerate(a):
        while st and a[st[-1]] < x:
            ans[st.pop()] = i + 1
        st.append(i)
    return ans

```

2.3 双指针 / 滑动窗口

题型：无重复字符最长子串

经典题目：LC3

```

def longest_no_repeat(s):
    seen = set()
    l = ans = 0
    for r, ch in enumerate(s):
        while ch in seen:
            seen.remove(s[l])
            l += 1
        seen.add(ch)
        ans = max(ans, r - l + 1)
    return ans

```

2.4 前缀和

前缀和模板

```

from itertools import accumulate

nums = [1,2,3,4,5]

pre = [0] + list(accumulate(nums))

# 区间和
def query(l,r):
    return pre[r+1] - pre[l]

```

技巧

accumulate 还能做 前缀最大值 / 最小值 / 乘积：

```

from itertools import accumulate
import operator

nums = [3,1,5,2,4]

pre_max = list(accumulate(nums, max))
## [3,3,5,5,5]

pre_mul = list(accumulate(nums, operator.mul)) ## 前缀乘积

```

3. 字符串

3.1 基础

- 常见：计数/哈希/双指针。

3.2. 字符串进阶 (替代 KMP 方案)

3.2.1 倒排索引与多词查询 (Inverted Index)

题型：给定多个单词出现的文档列表，处理“必须包含/必须排除”的组合查询。

核心：利用 `set` 的交集(`&`)、并集(`|`)、差集(`-`) 极速模拟逻辑门，切忌用 `list` 循环。

```

def inverted_index_query(query_flags, word_docs):
    # query_flags: 长度为N的列表，1表示包含，-1表示排除，0表示无所谓
    # word_docs: 长度为N的列表，每个元素是一个 set，存有该词出现的文档编号

    include_sets = [word_docs[i] for i in range(len(query_flags)) if query_flags[i] == 1]
    exclude_sets = [word_docs[i] for i in range(len(query_flags)) if query_flags[i] == -1]

    if not include_sets:
        return []

    # 1. 对所有“必须包含”的集合求交集
    # 注意: set.intersection(*list_of_sets) 是极其高效的写法
    valid_docs = set.intersection(*include_sets)

    # 2. 对所有“必须排除”的集合求并集，并从交集结果中减去
    if exclude_sets:
        invalid_docs = set.union(*exclude_sets)
        valid_docs -= invalid_docs

    return sorted(list(valid_docs))

```

3.2.2 最小循环节 / 字符串乘方 (Minimum Cycle)

题型：判断字符串是否由重复子串构成，求最短重复周期。替代 KMP 最小循环元引理。**原理：**若 `s` 由 `t` 重复构成，`s` 必然在 `(s + s)` 的错位切片中被提前找到。

```

def find_minimum_period(s: str) -> int:
    n = len(s)
    # 在 s+s 中从索引 1 开始找 s
    # 找到的第一个索引就是最小循环节的长度
    idx = (s + s).find(s, 1)

    if idx != -1 and idx != n:

```

```

return idx      # idx 是最小循环节的长度 (最大重复次数 K = n // idx)
return n        # 没有循环节, 整个字符串即为最小单位

```

3.2.3 Python 原生字符串高阶技巧 (Native String Ops)

说明: Python 的 `in`, `.find()`, `.count()` 底层由 C 语言通过 Boyer-Moore 算法高度优化。绝大多数常规字符串匹配题, 直接调用原生方法耗时极短, 应作为机考首选。

```

s = "abacab"
sub = "bac"

idx = s.find(sub)      # 找子串首个位置, 找不到返回 -1, 优于手写KMP
cnt = s.count(sub)     # 找子串不重叠出现的次数
is_prefix = s.startswith(sub) # 前缀匹配判断

```

4. 树

- 速记: 前序 根左右 / 中序 左根右 / 后序 左右根 / 层序 BFS。

4.1 遍历

题型: 前序 / 中序 / 层次

经典题目: 27638 二叉树高度与叶子

```

from collections import deque

class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

def preorder(root):
    if not root:
        return []
    return [root.val] + preorder(root.left) + preorder(root.right)

def level_order(root):
    if not root:
        return []
    q = deque([root])
    out = []
    while q:
        node = q.popleft()
        out.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    return out

```

4.2 建树

题型: 前序 + 中序

经典题目: 03720 文本二叉树

```

def build(pre, ino):
    if not pre:

```

```

    return None
root = TreeNode(pre[0])
k = ino.index(pre[0])
root.left = build(pre[1:1 + k], ino[:k])
root.right = build(pre[1 + k:], ino[k + 1:])
return root

```

表示法

- 括号嵌套树：用栈或递归解析。
- 多叉树可转为“左孩子-右兄弟”二叉表示。

4.3 解析树 / AST（后缀表达式建树）

```

class Node:
    __slots__ = ("val", "left", "right")
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def build_postfix(tokens):
    st = []
    for t in tokens:
        if t.isdigit():
            st.append(Node(t))
        else:
            r = st.pop()
            l = st.pop()
            st.append(Node(t, l, r))
    return st[-1]

```

4.4 堆 / Huffman

题型：最小合并代价

```

import heapq

def huffman_cost(a):
    heapq.heapify(a)
    cost = 0
    while len(a) > 1:
        x = heapq.heappop(a)
        y = heapq.heappop(a)
        cost += x + y
        heapq.heappush(a, x + y)
    return cost

```

4.5 BST

题型：插入 / 查询

经典题目：BST 第K小

```

def bst_insert(root, x):
    if not root:
        return TreeNode(x)
    if x < root.val:
        root.left = bst_insert(root.left, x)

```



```

elif x > root.val:
    root.right = bst_insert(root.right, x)
return root

```

4.6 字典树 (Trie)

题型：前缀匹配 / 字符串搜索（如 OJ 按标题搜索）

说明：参考 Tree 讲义，Trie 的核心是每个节点指向子节点（自上而下），关注“路径构成的字符串”（与并查集自下而上的父指针相反）。机考中推荐使用 dict 嵌套结构，无需额外定义节点类，代码极其精炼。

```

class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, word: str):
        # 沿着路径插入字符，自上而下
        cur = self.root
        for char in word:
            if char not in cur:
                cur[char] = {}
            cur = cur[char]
        cur['#'] = True # '#' 标志单词结束

    def search(self, word: str) -> bool:
        # 完全匹配
        cur = self.root
        for char in word:
            if char not in cur:
                return False
            cur = cur[char]
        return '#' in cur

    def starts_with(self, prefix: str) -> bool:
        # 前缀匹配
        cur = self.root
        for char in prefix:
            if char not in cur:
                return False
            cur = cur[char]
        return True

```

5. 并查集 DSU

5.1 合并集合

题型：连通块 / 排队

经典题目：20169 排队 / 01611 The Suspects

```

''' 递归版：路径压缩 + 按秩 (rank) 合并    默认序号 1...n    '''
class DSURecursive:
    def __init__(self, n):
        self.fa = list(range(n + 1))
        self.rank = [0] * (n + 1)

    def find(self, x):
        if self.fa[x] != x:
            self.fa[x] = self.find(self.fa[x])
        return self.fa[x]

```

```
def union(self, x, y):
    x = self.find(x)
    y = self.find(y)
    if x == y:
        return False
    if self.rank[x] < self.rank[y]:
        x, y = y, x
    self.fa[y] = x
    if self.rank[x] == self.rank[y]:
        self.rank[x] += 1
    return True
```

```
''' 非递归版：路径压缩（两次遍历） '''
class DSUIterative:
    def __init__(self, n):
        self.fa = list(range(n + 1))

    def find(self, x):
        root = x
        while self.fa[root] != root:
            root = self.fa[root]
        # 路径压缩
        while self.fa[x] != x:
            parent = self.fa[x]
            self.fa[x] = root
            x = parent
        return root
```

6. 图

6.1 图表示 + BFS/DFS

题型：连通性 / 最短步数

经典题目：02815 城堡问题 / 3629 质数传送

- 应用：迷宫 / 词梯 / 连通块 / 无权最短路。

```
from collections import deque

def bfs(g, s):
    n = len(g)
    dist = [-1] * n
    prev = [-1] * n
    dist[s] = 0
    q = deque([s])
    while q:
        u = q.popleft()
        for v in g[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                prev[v] = u
                q.append(v)
    return dist, prev

def dfs(g, u, vis):
```

```
vis[u] = True
for v in g[u]:
    if not vis[v]:
        dfs(g, v, vis)
```

- 多源 BFS：所有源点同时入队，`dist = 0`。

```
def has_cycle_directed(g):
    n = len(g)
    color = [0] * n

    def dfs(u):
        color[u] = 1
        for v in g[u]:
            if color[v] == 1:
                return True
            if color[v] == 0 and dfs(v):
                return True
        color[u] = 2
        return False

    return any(color[i] == 0 and dfs(i) for i in range(n))
```

6.2 最短路 Dijkstra

题型：带权最短路

经典题目：05443 兔子与樱花

- 选择规则：无权 BFS；非负权 Dijkstra；含负权 Bellman-Ford；多源 Floyd。

```
import heapq

def dijkstra(g, s):
    n = len(g)
    INF = 10 ** 18
    dist = [INF] * n
    dist[s] = 0
    pq = [(0, s)]
    while pq:
        d, u = heapq.heappop(pq)
        if d != dist[u]:
            continue
        for v, w in g[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return dist
```

6.2.1 Bellman-Ford（含负权/判负环）

```
def bellman_ford(n, edges, s):
    INF = 10 ** 18
    dist = [INF] * n
    dist[s] = 0
    for _ in range(n - 1):
        changed = False
```

```

for u, v, w in edges:
    if dist[u] != INF and dist[u] + w < dist[v]:
        dist[v] = dist[u] + w
        changed = True
if not changed:
    break
neg_cycle = any(dist[u] != INF and dist[u] + w < dist[v] for u, v, w in edges)
return dist, neg_cycle

```

6.2.2 Floyd-Warshall (多源最短路)

```

def floyd(d):
    n = len(d)
    for k in range(n):
        for i in range(n):
            if d[i][k] == 10 ** 18:
                continue
            for j in range(n):
                nd = d[i][k] + d[k][j]
                if nd < d[i][j]:
                    d[i][j] = nd
    return d

```

6.3 拓扑排序

题型：DAG 依赖

经典题目：29702 二叉的水管

```

def topo(n, g, indeg):
    q = deque([i for i in range(n) if indeg[i] == 0])
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else None

```

6.4 MST Kruskal

题型：最小生成树

经典题目：05442 兔子与星空

```

def kruskal(n, edges):
    # 功能：返回无向带权图的最小生成树权值和
    edges.sort()
    dsu = DSU(n)
    total = 0
    for w, u, v in edges:
        if dsu.union(u, v):
            total += w
    return total

```

6.5 MST Prim

题型：最小生成树

功能：从任意起点扩展，返回无向连通带权图的最小生成树权值和

```
import heapq

def prim(n, g, start=0):
    # 输入: n 个点, 邻接表 g[u] = [(v, w), ...]
    # 输出: MST 总权值 (假设图连通)
    vis = [False] * n
    pq = [(0, start)]
    total = 0
    picked = 0
    while pq and picked < n:
        w, u = heapq.heappop(pq)
        if vis[u]:
            continue
        vis[u] = True
        total += w
        picked += 1
        for v, cost in g[u]:
            if not vis[v]:
                heapq.heappush(pq, (cost, v))
    return total
```

6.6 SCC Tarjan (模板)

题型：强连通分量

功能：通用邻接表版本，输出所有 SCC (每个 SCC 内已排序)

```
import sys

sys.setrecursionlimit(200000)

def tarjan_scc(graph):
    # 输入: graph 为 1..n 的邻接表 (graph[0] 空)
    # 输出: 按最小点编号排序的 SCC 列表
    n = len(graph)
    timer = 0
    dfn = [-1] * n
    low = [-1] * n
    on_stack = [False] * n
    stack = []
    sccs = []

    def dfs(u):
        nonlocal timer
        dfn[u] = low[u] = timer
        timer += 1
        on_stack[u] = True
        stack.append(u)
        for v in graph[u]:
            if dfn[v] == -1:
                dfs(v)
                low[u] = min(low[u], low[v])
            elif on_stack[v]:
                low[u] = min(low[u], dfn[v])
        if dfn[u] == low[u]:
            tmp = []
            while True:
                v = stack.pop()
                on_stack[v] = False
                tmp.append(v)
            sccs.append(tmp)

    for i in range(1, n):
        if dfn[i] == -1:
            dfs(i)
```

```

        while stack:
            top = stack.pop()
            tmp.append(top)
            on_stack[top] = False
            if top == u:
                break
            sccs.append(sorted(tmp))

    for i in range(1, n):
        if dfn[i] == -1:
            dfs(i)
    return sorted(sccs, key=lambda x: x[0])

def main():
    it = iter(sys.stdin.read().split())
    n, m = int(next(it)), int(next(it))
    graph = [[] for _ in range(n + 1)]
    for _ in range(m):
        u, v = int(next(it)), int(next(it))
        graph[u].append(v)
    sccs = tarjan_scc(graph)
    print(len(sccs))
    for scc in sccs:
        print(*scc, sep=' ')

if __name__ == '__main__':
    main()

```

6.7 AOE 关键路径

- DAG 上拓扑排序。
- 计算最早/最晚发生时间，松弛时间为 0 的活动是关键活动。

```

def topo_sort(n, G, in_degree):
    """
    执行拓扑排序并计算每个事件的最早开始时间 ve
    """
    # 查找所有入度为 0 的点作为起点
    q = deque([i for i in range(n) if in_degree[i] == 0])
    ve = [0] * n
    topo_order = []

    while q:
        u = q.popleft()
        topo_order.append(u)
        for edge in G[u]:
            v = edge.v
            # 更新最早开始时间: ve[v] = max(ve[v], ve[u] + weight)
            if ve[u] + edge.w > ve[v]:
                ve[v] = ve[u] + edge.w

            in_degree[v] -= 1
            if in_degree[v] == 0:
                q.append(v)

    # 判环: 如果拓扑序列长度不等于 N, 说明有环 (AOE网不能有环)
    if len(topo_order) != n:

```

```

        return ve, topo_order
    else:
        return None, None

def get_critical_path(n, G, in_degree):
    """
    计算关键路径的核心逻辑
    """
    # 步骤 1: 拓扑排序求 ve (最早发生时间)
    ve, topo_order = topo_sort(n, G, in_degree)
    if ve is None:
        return -1, None

    # 项目总工期即为 ve 中的最大值
    maxLength = max(ve)

    # 步骤 2: 反向推导求 vl (最晚发生时间)
    # 初始化所有节点的最晚发生时间为总工期
    vl = [maxLength] * n

    # 按照拓扑序列的逆序进行更新
    for u in reversed(topo_order):
        for edge in G[u]:
            v = edge.v
            # 更新最晚开始时间: vl[u] = min(vl[u], vl[v] - weight)
            if vl[v] - edge.w < vl[u]:
                vl[u] = vl[v] - edge.w

    # 步骤 3: 寻找关键活动并构建关键图
    # 关键活动定义: 活动的 e[i] == l[i]
    # 即对应边 <u, v> 满足: ve[u] == vl[v] - weight
    activity = defaultdict(list)
    for u in range(n):
        for edge in G[u]:
            v = edge.v
            e = ve[u] # 活动最早开始时间
            l = vl[v] - edge.w # 活动最晚开始时间
            if e == l:
                activity[u].append(v)

    return maxLength, activity

```

7. DP / 回溯 / 贪心

7.1 0/1 背包

题型：容量限制最大价值

经典题目：背包类

```

def knap01(V, cost, val):
    # 倒序遍历容量，保证每件物品只用一次
    dp = [0] * (V + 1)
    for c, v in zip(cost, val):
        for j in range(V, c - 1, -1):
            dp[j] = max(dp[j], dp[j - c] + v)
    return dp[V]

```

7.2 回溯

题型：全排列 / 子集

经典题目：02488 Knight's Journey

```
def permute(nums):
    n = len(nums)
    used = [False] * n
    ans = []
    path = []

    def dfs():
        if len(path) == n:
            ans.append(path[:])
            return
        for i in range(n):
            if used[i]:
                continue
            used[i] = True
            path.append(nums[i])
            dfs()
            path.pop()
            used[i] = False

    dfs()
    return ans
```

7.3 贪心

题型：区间调度

经典题目：02442 Sequence

```
def max_nonoverlap(intervals):
    # 按结束时间排序，优先选最早结束的区间
    intervals.sort(key=lambda x: x[1])
    cnt = 0
    end = -10 ** 18
    for l, r in intervals:
        if l >= end:
            cnt += 1
            end = r
    return cnt
```

8. 可选模板

8.1 树状数组 BIT

题型：单点更新 + 前缀和

经典题目：M307 区域和检索

```
class BIT:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)

    def add(self, i, delta):
        while i <= self.n:
            self.bit[i] += delta
            i += i & -i
```



```

def sum(self, i):
    res = 0
    while i > 0:
        res += self.bit[i]
        i &= i - 1
    return res

def range_sum(self, l, r):
    return self.sum(r) - self.sum(l - 1)

```

8.2 线段树 Segment Tree (区间加 + 区间和)

题型：区间加法 + 区间查询

功能：维护区间和，支持区间加、区间查询 (含 lazy_tag)

```

class SegTree:
    def __init__(self, a):
        self.n = len(a)
        self.seg = [0] * (4 * self.n)
        self.lazy = [0] * (4 * self.n)
        self._build(1, 0, self.n - 1, a)

    def _build(self, idx, l, r, a):
        if l == r:
            self.seg[idx] = a[l]
            return
        mid = (l + r) // 2
        self._build(idx * 2, l, mid, a)
        self._build(idx * 2 + 1, mid + 1, r, a)
        self.seg[idx] = self.seg[idx * 2] + self.seg[idx * 2 + 1]

    def _apply(self, idx, l, r, delta):
        self.seg[idx] += delta * (r - l + 1)
        self.lazy[idx] += delta

    def _push(self, idx, l, r):
        if self.lazy[idx] != 0 and l != r:
            mid = (l + r) // 2
            self._apply(idx * 2, l, mid, self.lazy[idx])
            self._apply(idx * 2 + 1, mid + 1, r, self.lazy[idx])
            self.lazy[idx] = 0

    def range_add(self, ql, qr, delta):
        # 功能: 给区间 [ql, qr] 全部加上 delta
        self._range_add(1, 0, self.n - 1, ql, qr, delta)

    def _range_add(self, idx, l, r, ql, qr, delta):
        if ql <= l and r <= qr:
            self._apply(idx, l, r, delta)
            return
        self._push(idx, l, r)
        mid = (l + r) // 2
        if ql <= mid:
            self._range_add(idx * 2, l, mid, ql, qr, delta)
        if qr > mid:
            self._range_add(idx * 2 + 1, mid + 1, r, ql, qr, delta)
        self.seg[idx] = self.seg[idx * 2] + self.seg[idx * 2 + 1]

```

```
def query(self, ql, qr):
    # 功能: 查询区间 [ql, qr] 的和
    return self._query(1, 0, self.n - 1, ql, qr)

def _query(self, idx, l, r, ql, qr):
    if ql <= l and r <= qr:
        return self.seg[idx]
    self._push(idx, l, r)
    mid = (l + r) // 2
    res = 0
    if ql <= mid:
        res += self._query(idx * 2, l, mid, ql, qr)
    if qr > mid:
        res += self._query(idx * 2 + 1, mid + 1, r, ql, qr)
    return res
```

9. 数学相关

欧式筛法

筛选素数

```
def euler_sieve(n):
    is_prime=[True]*(n+1)
    is_prime[0]=is_prime[1]=False
    primes=[]
    for i in range(2,n+1):
        if is_prime[i]:
            primes.append(i)
            for p in primes:
                if i*p>n:
                    break
                is_prime[i*p]=False
                if i%p==0:
                    break
    return primes
```

分解质因数

```
MX=1000001
prime_factors=[[ ] for _ in range(MX)]
for i in range(2,MX):
    if not prime_factors[i]:
        for j in range(i,MX,i):
            prime_factors[j].append(i)
```

康托展开

计算排列方式的顺序数

对于 $P = [a_1, a_2, \dots, a_n]$ 中全部元素的一种排列，其康托展开值 x 的计算公式为：

$$x = a_1 (n-1)! + a_2 (n-2)! + \dots + a_{n-1} 1! + a_n 0!$$

其中 a_i 是该位数字在剩余未使用数字中的排名（从0开始）

Catalan 卡特兰数

应用场景：有多少个合法出栈序列，合法括号序列数量，满节点二叉树数量，凸多边形的三角划分种类数。

```

import math

# =====
# 知识点速查: 卡特兰数 (Catalan Number)
#
# 基础定义:
# C_0 = 1
# C_{n+1} = sum(C_i * C_{n-i}) (0 <= i <= n)
#
# 常见应用场景 (如果题目让你求以下数量, n 为基数, 答案均为 C_n):
# 1. n 对括号的合法序列数 (如 n=3 时有 5 种: ((())), ()(()), ()()(), (())(), (()()) )
# 2. n 个节点组成的不同二叉搜索树 (BST) 的种数
# 3. 拥有 n 个非叶子节点的满二叉树形态数
# 4. n+2 边凸多边形的无交集三角划分数
# 5. n 个元素进栈后, 不同的出栈序列数
# =====

class Catalan:

    def calc_dp(n: int) -> int:
        """ 方法一: 动态规划 / 递推公式 """
        if n <= 1:
            return 1

        dp = [0] * (n + 1)
        dp[0] = 1
        dp[1] = 1

        for i in range(2, n + 1):
            for j in range(i):
                dp[i] += dp[j] * dp[i - 1 - j]
        return dp[n]

    def calc_math(n: int) -> int:
        """ 方法二: 组合数直接计算 """
        if n < 0:
            return 0

        return math.comb(2 * n, n) // (n + 1)

```

最大公约数GCD和最小公倍数LCM

```

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

```